

MULTIVIEW: A Portable C99 System for Two-Camera Scene Analysis and 3D Model Extraction

A self-contained overview of the system and its mathematics

Project documentation — v0.2.0

August 2, 2026

Abstract

MULTIVIEW is a small, dependency-free C99 library for metric scene analysis and 3D model extraction from the synchronized video streams of *two stationary cameras* observing an overlapping volume, optionally augmented by a collection of still photographs and, in a later stage, range-finder measurements. Calibration is part of the system: the rig is calibrated from views of a printed letter-page checkerboard of known square size, or from a computer display under our control showing synchronized Gray-code patterns. This document doubles as a self-study overview of the underlying mathematics: each concept is developed from first principles, cross-linked to the modules that implement it, illustrated by a *hand-computable* worked example, and sourced in the bibliography. All quantitative claims ([section 13](#)) are reproducible from the deterministic test programs in the repository.

Contents

1	Introduction and capabilities	3
2	System overview	4
3	Mathematical toolkit	4
3.1	Homogeneous coordinates	4
3.2	Rotations	4
3.3	The cross-product matrix	5
3.4	Homogeneous least squares and the SVD	5
4	Camera model	5
5	Calibration from planar targets	6
5.1	The homography of a plane	6
5.2	Zhang’s constraints and the closed-form intrinsics	6
5.3	Radial distortion and alternation	7
5.4	The printed target	7
5.5	The display as an active target	8
5.6	Design: the rotating display	8
6	Two-view epipolar geometry	9

7	Triangulation	10
8	Rectification and dense stereo	11
9	From depth to models	12
9.1	The representation ladder	12
9.2	Background extraction: the easiest product	12
9.3	Separating the movers	13
10	Scaling the rig: more cameras and more pixels	13
10.1	N cameras: accuracy	13
10.2	N cameras: occlusion and topology	14
10.3	Resolution	15
10.4	Unequal resolutions	15
11	Planned extensions	16
11.1	Auxiliary photo collections	16
11.2	Range-finder fusion (optional, kept open)	16
12	Numerical implementation notes	17
13	Basic results	17
13.1	Error budget and consistency: is there a bug?	19
13.2	Statistical decision rules: when the numbers would say “bug”	21
14	Scenario studies	22
14.1	Insect counting and entry/exit statistics	22
14.2	Diurnal cycle and artificial lighting	22
14.3	Human occupancy	23
14.4	What the scenarios share	23
15	Person detection and face recognition with open-source components	23
15.1	Component landscape	23
15.2	Piping it in	24
15.3	Pre-registered vs unknown faces	24
15.4	Plugin matrix: OSS solutions, features, and license isolation	25
15.5	Deployment architecture: hardware boundaries and communication	26
16	Roadmap	26
A	Camera hardware and the quality of data	27
A.1	Shutter: global vs rolling	27
A.2	Synchronization: hardware trigger vs free-running	28
A.3	Interface: UVC vs machine-vision stacks	28
A.4	Optics: M12 vs C/CS mounts	28
A.5	Monochrome vs color	29
A.6	Summary and candidate hardware	29
A.7	Blending heterogeneous cameras	30

B	Root-mean-square (RMS) error: definition and use	31
B.1	Definition	31
B.2	Why squares	31
B.3	Bias, variance, and what RMS cannot tell you	31
B.4	Statistical handling	32
B.5	Where this document uses RMS	32

1 Introduction and capabilities

The target deployment is deliberately narrow and fully under our control: two cameras are rigidly mounted and observe the same working volume. The system first *calibrates itself* from views of a known planar target ([section 5](#)); thereafter video is processed frame by frame. Because the cameras do not move, every geometric quantity that depends only on the rig — projection matrices, the fundamental matrix, the rectifying homographies — is computed *once* and reused for every frame, which is the central efficiency win of the stationary-rig assumption.

The implemented core (v0.2.0) provides:

1. plane-based calibration of intrinsics, distortion, and pose from a printed letter-page checker-board or from a controlled display showing Gray-code patterns ([section 5](#));
2. a calibrated pinhole camera model with Brown radial-tangential lens distortion, forward projection and ray back-projection ([section 4](#));
3. exact two-view epipolar geometry from the calibration, and the normalized 8-point algorithm to estimate or verify it from image correspondences ([section 6](#));
4. N -view linear triangulation of corresponding image points into metric 3D, with reprojection-error reporting ([section 7](#));
5. planar stereo rectification of the pair, and dense correspondence by block matching with sub-pixel refinement, giving per-pixel depth ([section 8](#));
6. point-cloud assembly and PLY export for inspection in standard mesh tools, plus minimal PGM image I/O ([section 9](#)).

Planned extensions — structure-from-motion over an auxiliary photo collection and fusion of range-finder data — are designed but not yet implemented; their theory and integration points are described in [section 11](#) so that the code can grow into them without rearchitecting.

How to read this document. [section 3](#) collects the small amount of linear algebra everything else stands on. [section 4–section 8](#) then follow the data through the system: camera model, calibration, two-view geometry, triangulation, dense stereo. Every section ends in a worked example with numbers chosen so the computation fits on paper; doing them by hand *is* the intended self-study exercise, and each can then be checked against the library.

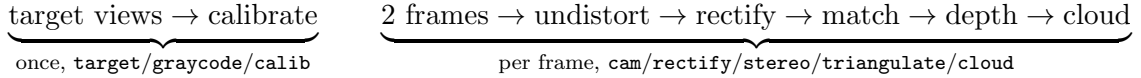
Design constraints. The library is strict C99 with no dependencies beyond `libm`, row-major `double` arrays throughout, and fully deterministic (no clocks, no ambient randomness; the demos use fixed-seed generators). This keeps it portable from desktop to embedded targets, and keeps every experiment repeatable — a prerequisite for the controlled-environment, learning-oriented use the project is intended for.

2 System overview

Module	Header	Responsibility
<code>mat</code>	<code>mv/mat.h</code>	small dense linear algebra, Jacobi SVD, null vectors
<code>cam</code>	<code>mv/cam.h</code>	camera model, projection, rays, distortion
<code>target</code>	<code>mv/target.h</code>	printed checkerboard model + printable rendering
<code>graycode</code>	<code>mv/graycode.h</code>	display-based active target (Gray-code patterns)
<code>calib</code>	<code>mv/calib.h</code>	homographies; Zhang calibration; radial estimate
<code>epipolar</code>	<code>mv/epipolar.h</code>	$\mathbf{F}$ from calibration; normalized 8-point; $\mathbf{E}$
<code>triangulate</code>	<code>mv/triangulate.h</code>	N -view DLT triangulation, reprojection RMS
<code>rectify</code>	<code>mv/rectify.h</code>	Fusiello rectification, disparity→depth
<code>stereo</code>	<code>mv/stereo.h</code>	dense block matching (SAD, sub-pixel)
<code>img</code>	<code>mv/img.h</code>	PGM (P5) image I/O
<code>cloud</code>	<code>mv/cloud.h</code>	point cloud container, PLY export

Table 1: Module layout. `mv/mv.h` is the umbrella header.

Processing has a one-time phase and a per-frame phase:



Sparse operation (feature correspondences instead of dense matching) uses the same chain with `epipolar` providing the correspondence-validation gate. Because the rig is static, temporal accumulation over video frames is a simple union of per-frame clouds in the fixed world frame; moving objects appear as temporally inconsistent points and can be segmented by comparing frames — the entry point for scene *analysis* beyond reconstruction.

3 Mathematical toolkit

Four ideas carry the whole system; everything later is an application.

3.1 Homogeneous coordinates

A 2D point (u, v) is represented as any triple (wu, wv, w) , $w \neq 0$; a 3D point (X, Y, Z) as $(X, Y, Z, 1)$ up to scale. The payoff: projection, homographies and rigid motions all become *linear* maps, and points at infinity ($w = 0$) need no special case. We write $\simeq$ for equality up to a nonzero scale.

Example 1 (dehomogenization). $(120, 140, 2) \simeq (60, 70, 1)$: both name the pixel $(60, 70)$. Conversely the direction “straight down the u axis” is $(1, 0, 0)$, which no finite pixel represents.

3.2 Rotations

A rotation matrix $\mathbf{R} \in SO(3)$ satisfies $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$; its rows are the axes of the new frame expressed in the old. Consequently $\mathbf{R}^{-1} = \mathbf{R}^\top$ — inverting a rigid transform never requires a matrix inversion.

Example 2 (rotation about y). $\mathbf{R}_y(90^\circ) = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$ sends the world z axis $(0, 0, 1)$ to $(-1, 0, 0)$: a camera yawed 90° sees the world's $-x$ direction ahead. Check $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ by inspection: the rows are orthonormal.

3.3 The cross-product matrix

For $\mathbf{v} = (v_1, v_2, v_3)$,

$$[\mathbf{v}]_\times = \begin{pmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & -v_1 \\ -v_2 & v_1 & 0 \end{pmatrix}, \quad [\mathbf{v}]_\times \mathbf{w} = \mathbf{v} \times \mathbf{w}. \quad (1)$$

It is rank 2 with null vector $\mathbf{v}$ itself — the algebraic reason epipoles exist (section 6).

Example 3 (skew matrix). $\mathbf{v} = (1, 0, 0)$: $[\mathbf{v}]_\times = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$, and $[\mathbf{v}]_\times (0, 1, 0)^\top = (0, 0, 1)^\top = \mathbf{v} \times \mathbf{w}$.

3.4 Homogeneous least squares and the SVD

Every estimation problem in this system reduces to: given $\mathbf{A}$ ($m \times n$), find $\mathbf{x} \neq 0$ minimizing $\|\mathbf{Ax}\|$ subject to $\|\mathbf{x}\| = 1$. The minimizer is the right singular vector of the smallest singular value of $\mathbf{A} = \mathbf{U} \text{diag}(\sigma_1 \geq \dots \geq \sigma_n) \mathbf{V}^\top$ [GV13]. The library computes this with a one-sided Jacobi SVD (section 12) via `mv_nullvec`.

Example 4 (null vector by hand). $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ has rank 1 (row 2 = $2 \times$ row 1), so the smallest singular value is exactly 0 and the null vector solves $x_1 + 2x_2 = 0$: $\mathbf{x} = \frac{1}{\sqrt{5}}(2, -1)$. Check: $\mathbf{Ax} = \mathbf{0}$.

4 Camera model

A camera is a pinhole with intrinsic matrix

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad (2)$$

rotation $\mathbf{R}$ and translation $\mathbf{t}$ mapping world points into the camera frame, $\mathbf{X}_c = \mathbf{RX} + \mathbf{t}$; the camera center is $\mathbf{C} = -\mathbf{R}^\top \mathbf{t}$. A world point projects to the homogeneous pixel

$$\mathbf{x} \simeq \mathbf{P} \begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix}, \quad \mathbf{P} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}] \quad (3)$$

[HZ04]. Real lenses deviate from (3); on normalized coordinates $(x, y) = (X_c/Z_c, Y_c/Z_c)$ with $r^2 = x^2 + y^2$ the Brown model gives the distorted

$$\begin{aligned} x_d &= x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \\ y_d &= y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \end{aligned} \quad (4)$$

and the pixel is $\mathbf{K}(x_d, y_d, 1)^\top$. Back-projection inverts (4) by fixed-point iteration and returns the ray $\mathbf{C} + s\mathbf{d}$, $s > 0$.

Example 5 (projection). Let $f_x=f_y=100$, $c_x=c_y=50$, $s=0$, $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = \mathbf{0}$, no distortion, and $\mathbf{X} = (1, 2, 10)$. Normalized coordinates: $(x, y) = (1/10, 2/10) = (0.1, 0.2)$. Pixel: $u = 100 \cdot 0.1 + 50 = 60$, $v = 100 \cdot 0.2 + 50 = 70$ — the point lands at $(60, 70)$, cf. [Example 1](#).

Example 6 (camera center). Let $\mathbf{R} = \mathbf{R}_y(90^\circ)$ from [Example 2](#) and $\mathbf{t} = (3, 0, 0)$. Then $\mathbf{R}^\top \mathbf{t} = (0, 0, -3)$ and $\mathbf{C} = -\mathbf{R}^\top \mathbf{t} = (0, 0, 3)$: the camera sits three units down the world z axis. Check forward: $\mathbf{RC} + \mathbf{t} = (0 \cdot 0 + 0 \cdot 0 - 1 \cdot 3, 0, 0 \cdot 3) + (3, 0, 0) = (-3, 0, 0) + (3, 0, 0) = \mathbf{0}$ — the center maps to the camera origin, as it must.

5 Calibration from planar targets

Calibration is part of the system, not an external input. Both supported targets are *planes with known metric structure*: a printed checkerboard page ([subsection 5.4](#)) and a controlled computer display ([subsection 5.5](#)). The mathematics is the same for both: several views of a known plane determine the intrinsics, the distortion, and each view’s pose, by Zhang’s method [[Zha00](#), [SM99](#)].

5.1 The homography of a plane

Put the target plane at $Z = 0$ in its own frame. Then (3) collapses: writing $\mathbf{R} = [\mathbf{r}_1 \mathbf{r}_2 \mathbf{r}_3]$ (columns),

$$\mathbf{x} \simeq \mathbf{K} [\mathbf{r}_1 \mathbf{r}_2 \mathbf{t}] (X, Y, 1)^\top = \mathbf{H} (X, Y, 1)^\top, \quad (5)$$

a 3×3 *homography* with 8 degrees of freedom. Each point correspondence gives two linear equations in the entries of $\mathbf{H}$, so $n \geq 4$ points determine it as a homogeneous least-squares problem ([Example 4](#) at scale $2n \times 9$), solved by normalized DLT (`mv_homography_dlt`; normalization as in [section 12](#)).

Example 7 (plane homography). Frontal camera: $\mathbf{K}$ as in [Example 5](#), $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = (0, 0, 2)$. Then $[\mathbf{r}_1 \mathbf{r}_2 \mathbf{t}] = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix}$ and

$$\mathbf{H} = \mathbf{K} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 100 & 0 & 100 \\ 0 & 100 & 100 \\ 0 & 0 & 2 \end{pmatrix} \simeq \begin{pmatrix} 50 & 0 & 50 \\ 0 & 50 & 50 \\ 0 & 0 & 1 \end{pmatrix}.$$

The plane point $(X, Y) = (0.1, 0.2)$ maps to $(50 \cdot 0.1 + 50, 50 \cdot 0.2 + 50) = (55, 60)$. Check via [Example 5](#): the 3D point is $(0.1, 0.2, 2)$, normalized $(0.05, 0.1)$, pixel $(100 \cdot 0.05 + 50, 100 \cdot 0.1 + 50) = (55, 60)$.

5.2 Zhang’s constraints and the closed-form intrinsics

Write $\mathbf{H} = [\mathbf{h}_1 \mathbf{h}_2 \mathbf{h}_3]$ (columns). From (5), $\mathbf{h}_1 \simeq \mathbf{K} \mathbf{r}_1$ and $\mathbf{h}_2 \simeq \mathbf{K} \mathbf{r}_2$ with the *same* scale. Since $\mathbf{r}_1, \mathbf{r}_2$ are orthonormal,

$$\mathbf{h}_1^\top \mathbf{B} \mathbf{h}_2 = 0, \quad \mathbf{h}_1^\top \mathbf{B} \mathbf{h}_1 = \mathbf{h}_2^\top \mathbf{B} \mathbf{h}_2, \quad \mathbf{B} := \mathbf{K}^{-\top} \mathbf{K}^{-1}, \quad (6)$$

i.e. each view of the plane contributes two linear equations in the six independent entries of the symmetric matrix $\mathbf{B}$ (the *image of the absolute conic*). Three views generically determine $\mathbf{B}$ up to scale; two suffice if zero skew ($s=0 \Leftrightarrow B_{12}=0$) is imposed. The solution is again a null

vector (Example 4); $\mathbf{K}$ then follows from $\mathbf{B}$ in closed form [Zha00], and the pose of each view is recovered as

$$\lambda = \frac{1}{\|\mathbf{K}^{-1}\mathbf{h}_1\|}, \quad \mathbf{r}_1 = \lambda\mathbf{K}^{-1}\mathbf{h}_1, \quad \mathbf{r}_2 = \lambda\mathbf{K}^{-1}\mathbf{h}_2, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \quad \mathbf{t} = \lambda\mathbf{K}^{-1}\mathbf{h}_3, \quad (7)$$

with $[\mathbf{r}_1 \mathbf{r}_2 \mathbf{r}_3]$ snapped to the nearest true rotation via SVD (noise makes it only approximately orthonormal).

Example 8 (pose from a homography, by hand). Take the (unscaled) $\mathbf{H}$ of Example 7 and the known $\mathbf{K}$. Then

$$\mathbf{K}^{-1} = \begin{pmatrix} 0.01 & 0 & -0.5 \\ 0 & 0.01 & -0.5 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_3 = \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix}.$$

Both constraint equations of (6) hold: $\mathbf{r}_1 \cdot \mathbf{r}_2 = 0$ and $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$. The scale is $\lambda = 1/\|(1, 0, 0)\| = 1$, so (7) gives $\mathbf{r}_1 = (1, 0, 0)$, $\mathbf{r}_2 = (0, 1, 0)$, $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 = (0, 0, 1)$, $\mathbf{t} = (0, 0, 2)$ — exactly the frontal pose two metres from the target that we started from.

5.3 Radial distortion and alternation

With $\mathbf{K}$ and poses in hand, the ideal (undistorted) projection (u, v) of every target point is known, and (4) predicts the observed (u_d, v_d) linearly in (k_1, k_2) :

$$u_d - u = (u - c_x)(k_1 r^2 + k_2 r^4), \quad v_d - v = (v - c_y)(k_1 r^2 + k_2 r^4), \quad (8)$$

a least-squares problem with two unknowns. Since the homographies were fit to *distorted* points, the library alternates: fit geometry, estimate (k_1, k_2) , undistort the observations, refit — a few rounds converge for moderate distortion [Zha00].

An identifiability caveat worth internalizing: over the small radius range a letter-page target covers, r^2 and r^4 are nearly proportional, so k_1 and k_2 are individually ill-determined even when their combined *curve* $k_1 r^2 + k_2 r^4$ — the thing that actually corrects pixels — is fit to well below noise (measured in section 13). Wide targets, or the full-field display target of subsection 5.5, are the cure when the individual coefficients matter.

5.4 The printed target

The standard passive target is a checkerboard printed on a letter/A4 page: 8×10 squares of nominally 24 mm (192×240 mm), i.e. 7×9 inner corners (`mv/target.h`; `mv_target_render_pgm` emits the printable image — at 10 px/mm a 24 mm square is 240 px). Printers rescale, so the procedure is *print, then measure one square* and pass the measured size; only that measurement makes the calibration metric. Corner *positions* in the image are currently supplied by the caller (an external detector or manual marking); a saddle-point corner detector is on the roadmap (section 16). Ten to twenty views with strong tilt diversity (± 30 – 40°) condition the focal length well — near-frontal views leave it weakly observable, a property visible in the experiment of section 13.

5.5 The display as an active target

A computer display we control is a better target than paper wherever one is available: its pixel grid is a plane of exactly known geometry (pixel pitch from the panel datasheet, or measured once), and because we control what it shows *when*, correspondence needs no detector at all. The display plays, synchronized with capture, a sequence of binary stripe patterns that encode each display pixel’s coordinate in *Gray code* [ISM84, Gen11]: bit b of the code is shown as stripes, followed by its inverse, for $b = 0..[\log_2 W]-1$, then the same for the y axis. A camera pixel observing the display reads its bit as “brighter in the pattern or in the inverse?” — a per-pixel comparison immune to illumination, display brightness and gamma — and after all frames has decoded *which display pixel it sees* (`mv/graycode.h`). This yields thousands of dense, uniformly spread correspondences per view; the same pipeline (subsection 5.1–subsection 5.3) consumes them, with plane coordinates = display pixel $\times$ pitch. Gray code rather than plain binary because adjacent stripes differ in exactly one bit: a threshold error at a stripe boundary displaces the decoded coordinate by at most one pixel instead of, e.g., 2^b .

Example 9 (Gray code by hand). Encode $v = 6$: binary 110; Gray = $110 \oplus 011 = 101$. Decode $g = 101$ by prefix-XOR: $b_2 = 1$, $b_1 = 1 \oplus 0 = 1$, $b_0 = 1 \oplus 0 \oplus 1 = 0$ — binary 110 = 6. Now flip the last Gray bit (a boundary misread): $g' = 100 \rightarrow$ binary 111 = 7, an error of exactly one position. In plain binary, flipping the top bit of 110 gives 010 = 2, an error of four.

5.6 Design: the rotating display

For an N -camera rig (section 10) the planned calibration fixture is the display of subsection 5.5 mounted on a plate rotating at constant angular velocity ω , so that the screen sweeps past every camera each revolution. This subsection records the design; implementation is a roadmap item (section 16).

What the rotation provides. Three things, all substantial. (i) *Tilt diversity per camera*: as the screen sweeps past, every frame shows the known plane at a different orientation — exactly the varied-tilt view set Zhang’s method needs, whose absence measurably degrades focal recovery (section 13). (ii) *One shared orbit*: every camera observes the same one-parameter family of poses — screen pose = mount offset composed with rotation about a fixed axis at constant ω . Fitting that orbit (axis, ω , phase, mount; ~ 10 parameters) jointly across cameras links *all* of them into one world frame even if no two ever see the screen simultaneously: the orbit itself is the common reference object, replacing pairwise shared views. (iii) *The display is the clock*: the screen broadcasts its frame index, so every captured image self-reports display time and cameras synchronize to the rotation model without a hardware trigger.

What the rotation forbids. The static Gray-code stack of subsection 5.5 requires one camera pixel to observe one display point across ~ 20 frames — impossible on a moving target. The pattern must therefore be *single-shot self-identifying*: every frame decodable alone. The design is layered:

- a checkerboard-type *saddle-point grid* as the metric layer — saddle locations are unbiased under perspective and under (moderate, symmetric) blur, unlike dot centroids, which both foreshortening and blur displace;

- *identity coding* embedded in the grid (marker bits in alternate squares, or an M-array whose every $k \times k$ window is unique), so partial and oblique views self-locate — most frames of a sweep are partial;
- a *frame-counter strip* along the border, Gray-coded *in time* (successive indices differ in one bit, cf. [Example 9](#)), providing the timestamp for the orbit model;
- *multi-scale structure* (a coarse grid nested with a fine one), since one display serves cameras at different distances and obliquities; strokes must span ≥ 3 –4 pixels in the worst camera at the most oblique usable pose;
- *binary black/white only*: LCD gamma and contrast shift off-axis, so gray-level codes degrade with viewing angle while binary features with per-frame identity do not.

Across frames, only the identity layer changes (counter strip; optionally alternating coarse/fine emphasis) — pose variation is already supplied by the rotation. At low ω the screen moves <1 pixel between consecutive frames, so pattern-plus-inverse in consecutive frames remains usable and restores the per-pixel contrast robustness of [subsection 5.5](#).

Example 10 (motion-blur budget). Image-space blur during exposure t_e is $\omega r f t_e / Z$ pixels, with r the lever arm from axis to screen edge. For the experimental rig’s numbers ($f = 800$ px, $Z = 4.7$ m) with $r = 0.25$ m and $\omega = 6$ rpm = 0.63 rad/s: the screen edge moves $0.63 \cdot 0.25 \cdot 800 / 4.7 \approx 27$ px/s, so a 5 ms exposure blurs 0.13 px — comfortably sub-pixel, and between frames at 30 fps the screen shifts 0.9 px, validating the consecutive-inverse-frame option. The formula is the design knob linking ω , exposure, and screen brightness.

One mechanical trap: cant the screen. A screen standing parallel to the rotation axis presents orientations that all differ by rotation about a *single* axis — a weak/singular family for plane-based calibration [[SM99](#)], the same disease as the measured near-frontal weakness. Mounting the screen tilted back 20–30° makes its normal sweep a cone instead of a circle, exercising two rotation axes; this one bracket angle is the most important mechanical detail of the fixture.

Pipeline. Gate frames by obliquity (roughly 15–55° off-normal: frontal frames are uninformative, grazing frames unmatchable); per-frame homographies from the self-identifying grid; per-camera Zhang intrinsics from the gated set; joint orbit fit for all extrinsics + axis + ω + phase — the natural first customer for the planned Levenberg–Marquardt stage.

6 Two-view epipolar geometry

Corresponding pixels of the two views are linked by the fundamental matrix $\mathbf{F}$ through the epipolar constraint

$$\mathbf{x}_2^\top \mathbf{F} \mathbf{x}_1 = 0, \quad (9)$$

with $\mathbf{F}$ of rank 2; the line $\ell_2 = \mathbf{F} \mathbf{x}_1$ is the locus in image 2 of possible matches of $\mathbf{x}_1$ ([Figure 1](#)). For *calibrated* views the essential matrix $\mathbf{E} = \mathbf{K}_2^\top \mathbf{F} \mathbf{K}_1 = [\mathbf{t}]_\times \mathbf{R}$ carries the same constraint in normalized coordinates [[LH81](#)]; note the reappearance of (1).

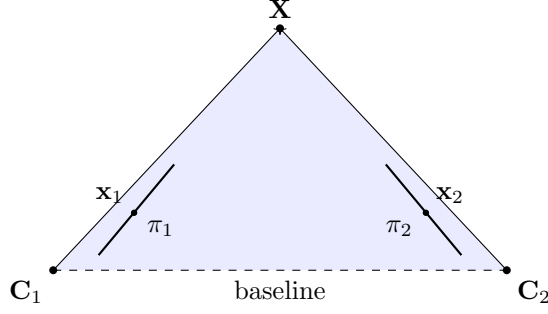


Figure 1: Epipolar geometry: the camera centers $\mathbf{C}_1, \mathbf{C}_2$ and the scene point $\mathbf{X}$ span the epipolar plane (shaded); its intersections with the image planes π_1, π_2 are the corresponding epipolar lines on which $\mathbf{x}_1$ and $\mathbf{x}_2$ must lie.

Because our rig is calibrated, $\mathbf{F}$ is available in closed form,

$$\mathbf{F} = [\mathbf{e}_2]_{\times} \mathbf{P}_2 \mathbf{P}_1^+, \quad \mathbf{e}_2 = \mathbf{P}_2 \begin{pmatrix} \mathbf{C}_1 \\ 1 \end{pmatrix}, \quad (10)$$

where $\mathbf{P}_1^+$ is the pseudo-inverse of $\mathbf{P}_1$ [HZ04]. The system computes (10) once at start-up and uses it for *correspondence gating*: any candidate match whose symmetric epipolar distance exceeds a threshold is rejected before triangulation.

Independently, $\mathbf{F}$ can be *estimated* from $n \geq 8$ correspondences by the normalized 8-point algorithm [Har97]: each pair turns (9) into one linear equation in the nine entries of $\mathbf{F}$ — again the homogeneous least-squares pattern of Example 4 — followed by rank-2 enforcement via SVD. In this system the estimator serves two purposes: (i) *health monitoring* — if the estimate from live correspondences drifts from the calibrated (10), the rig has been bumped or the calibration is stale; (ii) bootstrapping geometry for *uncalibrated* auxiliary photos (section 11). On contaminated correspondence sets the estimator is wrapped in a RANSAC loop [FB81] (planned with the feature layer).

Example 11 (essential matrix of a rectified rig). Two identical cameras, the second displaced one unit along x : $\mathbf{R} = \mathbf{I}$, $\mathbf{C}_2 = (1, 0, 0)$, so $\mathbf{t} = -\mathbf{R}\mathbf{C}_2 = (-1, 0, 0)$. By Example 3,

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}, \quad \mathbf{x}_2^{\top} \mathbf{E} \mathbf{x}_1 = (x_2, y_2, 1) \begin{pmatrix} 0 \\ 1 \\ -y_1 \end{pmatrix} = y_2 - y_1.$$

The epipolar constraint degenerates to $y_2 = y_1$: corresponding points lie on the *same image row*. This is precisely the situation rectification (section 8) manufactures for arbitrary rigs.

7 Triangulation

Given observations $\mathbf{x}^{(v)} = (u_v, w_v)$ of the same point in $N \geq 2$ views with projection matrices $\mathbf{P}^{(v)}$, each view contributes two linear equations obtained by eliminating the projective scale from (3):

$$\begin{pmatrix} u_v \mathbf{P}_{3,:}^{(v)} - \mathbf{P}_{1,:}^{(v)} \\ w_v \mathbf{P}_{3,:}^{(v)} - \mathbf{P}_{2,:}^{(v)} \end{pmatrix} \begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix} = \mathbf{0}, \quad (11)$$

and stacking all views gives a $2N \times 4$ homogeneous system solved, once more, as in [Example 4](#) — the DLT triangulation [[HZ04](#)]. The implementation accepts any $N \geq 2$, so still photographs registered into the rig frame ([section 11](#)) simply extend the same call; what additional views and additional pixels buy, quantitatively, is analyzed in [section 10](#). DLT minimizes an algebraic residual; the reprojection-optimal two-view method of Hartley–Sturm [[HS97](#)] is a planned refinement, worthwhile mainly for wide-baseline auxiliary views. The library reports the RMS reprojection error of every triangulated point — the quantity an eventual bundle adjustment would minimize [[TMHF00](#)].

Example 12 (triangulation by similar triangles). Rig of [Example 11](#) with $\mathbf{K}$ of [Example 5](#) ($f = 100$, $c = 50$) and baseline $B = 0.2$: camera 2 at $(0.2, 0, 0)$. Scene point $\mathbf{X} = (0.1, 0, 2)$. View 1: $(x, y) = (0.05, 0)$, pixel $u_1 = 100 \cdot 0.05 + 50 = 55$. View 2: camera coordinates $(0.1 - 0.2, 0, 2) = (-0.1, 0, 2)$, so $u_2 = 100 \cdot (-0.05) + 50 = 45$. Disparity $d = u_1 - u_2 = 10$, and (12) gives $Z = fB/d = 100 \cdot 0.2/10 = 2$ ✓. The lateral coordinate follows from view 1: $X = Z(u_1 - c_x)/f = 2 \cdot 5/100 = 0.1$ ✓ — the full point recovered by hand.

8 Rectification and dense stereo

For dense per-pixel depth the pair is first *rectified*: both cameras are rotated (synthetically, about their centers) onto a common orientation whose x axis is the baseline, making the geometry of [Example 11](#) true by construction — epipolar lines become identical image rows and correspondence search collapses to 1D. We use the compact algorithm of Fusiello, Trucco and Verri [[FTV00](#)]: the new rotation has rows $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ with $\mathbf{x}$ the unit baseline, $\mathbf{y} = \mathbf{k} \times \mathbf{x}$ ($\mathbf{k}$ the old optical axis of camera 1) and $\mathbf{z} = \mathbf{x} \times \mathbf{y}$; both views share the averaged intrinsics with zero skew, and the pixel maps are the homographies $\mathbf{H}_i = \mathbf{K}_{\text{new}} \mathbf{R}_{\text{new}} \mathbf{R}_i^\top \mathbf{K}_i^{-1}$ (homographies again — cf. (5), here induced by pure rotation instead of a scene plane). Since the rig is stationary, $\mathbf{H}_1, \mathbf{H}_2$ are computed once and each video frame is warped by table lookup.

On the rectified pair, dense correspondence is found by block matching: for each left pixel the sum of absolute differences (SAD) over a $(2w+1)^2$ window is minimized along the corresponding right row over a disparity range, followed by sub-pixel refinement by parabolic interpolation of the cost around the minimum. This is the classic local baseline method in the taxonomy of Scharstein and Szeliski [[SS02](#)] — chosen deliberately: it is simple, portable, trivially parallel, and its failure modes (textureless regions, occlusions, repetitive patterns) are well understood, giving a controlled baseline against which future matchers (rank/census transforms, semi-global aggregation) can be measured within the same harness.

Matched disparity d converts to metric depth by

$$Z = \frac{fB}{d}, \quad (12)$$

with f the shared rectified focal length (pixels) and B the baseline. Differentiating (12) gives the expected depth uncertainty

$$\sigma_Z = \frac{Z^2}{fB} \sigma_d \quad (13)$$

for disparity noise σ_d — the quadratic growth with Z is the fundamental accuracy law of the rig.

Example 13 (depth error budget). The experimental rig of [section 13](#): $f = 800$ px, $B = 0.5$ m, mean depth $Z = 4.7$ m, per-coordinate noise $\sigma = 0.3$ px so $\sigma_d = \sqrt{2}\sigma \approx 0.42$ px. Then $\sigma_Z = 4.7^2 \cdot 0.42 / (800 \cdot 0.5) \approx 0.023$ m: about 2.3 cm of depth uncertainty — compare the measured 2.65 cm RMS 3D error in [Table 2](#).

9 From depth to models

Every depth estimate back-projects to a 3D point in the fixed world frame; points are accumulated in a growable cloud with optional per-point color and exported as ASCII PLY, directly loadable in MeshLab, CloudCompare or Blender for inspection. This section maps out what can be built *on top* of that cloud: the representations worth targeting, how robust each is in the literature, and the background/foreground architecture the stationary rig makes natural.

9.1 The representation ladder

Model types form a ladder of increasing geometric commitment; each rung is derived from the one below, and robustness generally *decreases* upward:

1. *Point clouds* (implemented): no topology, no commitments; ideal for measurement and inspection; never wrong, never finished.
2. *Occupancy / signed-distance volumes*: the truncated signed distance function (TSDF) fuses many noisy depth maps by running per-voxel averages [\[CL96\]](#); incremental, noise-averaging, topology-agnostic — the workhorse of the KinectFusion lineage [\[NIH⁺11\]](#) and the natural accumulation structure for our per-frame depth.
3. *Triangulated meshes*: extracted from a TSDF by marching cubes [\[LC87\]](#), or from oriented points by Poisson reconstruction [\[KBH06\]](#). Both are mature and reliable *given* rung 2; meshing raw noisy stereo clouds directly (Delaunay-based, ball-pivoting) is fragile and best avoided.
4. *Geometric primitives* (“something more geometric”): RANSAC plane extraction [\[FB81\]](#) is the single most robust operation in the entire reconstruction literature — floors, walls, table tops fall out almost for free; cylinders, spheres and cones follow with efficient RANSAC variants [\[SWK07\]](#) at the cost of tuning; box/cuboid fits give furniture-level abstraction. Beyond quadrics, parametric fitting becomes brittle.
5. *Object/semantic models* (chair, table, person): robust today only via learned detectors — outside the dependency-free core, but the geometry below is exactly what such an external module would consume.

Modern learned implicit representations (neural radiance fields, Gaussian splatting) excel at appearance and view synthesis but are not metrically superior for measurement, and sit outside this system’s portable-C99, controlled-environment charter.

9.2 Background extraction: the easiest product

For a stationary rig, the *static scene* — walls, floor, furniture — is the most robustly extractable model, because time works entirely in its favor: the per-pixel *temporal median* of depth over

minutes or hours is immune to anything that moves through the scene (the same robust-statistics trick as classical 2D background subtraction, lifted to depth), and every additional frame averages noise down. The pipeline is: temporal-median depth map $\rightarrow$ TSDF fusion [CL96] $\rightarrow$ marching-cubes mesh [LC87] of the static scene $\rightarrow$ RANSAC planes for floor/walls/table-tops $\rightarrow$ primitive fits [SWK07] on the remaining clusters for furniture-level geometry. Every stage is rung-1–4 robust. The background model is also *self-maintaining*: a piece of furniture moved to a new position is a persistent change — detected exactly like a lamp event in section 14, but in geometry — triggering a local update and, as a by-product, an event log of scene rearrangements.

9.3 Separating the movers

With a background model in hand, foreground extraction is subtraction: any depth estimate (or triangulated point) that disagrees with the background beyond its noise band belongs to a *mover*. Foreground points are clustered by spatial proximity per frame, clusters are tracked over time (section 14 implements exactly this machinery), and classification into insect / robot / human / other falls out of crude but effective geometric statistics — extent, height above floor, speed, and persistence: a single isolated point moving at 0.1–1 m/s is an insect; a ~ 0.3 m-wide cluster at constant ~ 0.1 m height gliding on the floor plane is the robot; a 0.4–0.6 m cluster extending 1.5–2 m above the floor is a person. The scenario studies of section 14 quantify each of these tracks separately; the unified pipeline — background subtraction feeding one multi-target tracker with per-class models — is the integration goal they build toward.

10 Scaling the rig: more cameras and more pixels

The two-camera rig is the minimum that triangulates. This section works out what changes when the rig grows — N cameras around the same volume, or higher (possibly unequal) resolutions — so that hardware decisions can be made from the error laws instead of intuition. The library is already shaped for this: `mv_triangulate` accepts any $N \geq 2$, and every camera carries its own $\mathbf{K}$, so heterogeneous rigs need no new code paths in the sparse pipeline.

10.1 N cameras: accuracy

Adding cameras improves triangulation through two distinct mechanisms.

Geometry (the dominant effect). The depth law (13) is governed by the baseline: rays from nearby centers intersect at grazing angles, and the intersection point slides far along the rays for small image perturbations. With cameras spread out, the *widest* usable pair sets the leading term: for N collinear cameras spaced B apart, the widest baseline is $(N-1)B$ and

$$\sigma_Z^{\text{widest}} = \frac{Z^2}{f(N-1)B} \sigma_d, \quad (14)$$

an $(N-1)$ -fold improvement before any averaging. The catch is correspondence: wide pairs see the surface from very different angles, so matching is harder — which is exactly why multi-baseline stereo [OK93] matches *short* pairs (easy, unambiguous) and accumulates their evidence in a common inverse-depth parameterization, getting wide-pair accuracy at short-pair matching difficulty. Repetitive texture that fools one pair (a periodic fence aliases at one baseline) is

disambiguated by any other baseline whose period does not divide it — the match must be consistent across *all* pairs simultaneously.

Redundancy. With visibility and geometry held fixed, N views of the same point give the usual statistical $\sqrt{N}$: the DLT stack (11) grows to $2N \times 4$ rows and the estimate averages the per-view noise. This is the smaller effect — doubling N buys $\sqrt{2} \approx 1.4\times$; doubling the *spread* buys $2\times$.

Example 14 (four collinear cameras). Extend the experimental rig of Example 13 ($f = 800$ px, $Z = 4.7$ m, $\sigma_d = 0.42$ px) from two cameras 0.5 m apart to four cameras at 0, 0.5, 1.0, 1.5 m. The widest pair has $B = 1.5$ m, so by (14) $\sigma_Z = 4.7^2 \cdot 0.42 / (800 \cdot 1.5) \approx 7.7$ mm — three times better than the 23 mm of the original pair, from geometry alone; fusing the remaining pairs tightens it further. The price: the widest pair’s viewing directions differ by $2 \arctan(0.75/4.7) \approx 18^\circ$, and matching windows must survive that foreshortening — the Okutomi–Kanade argument for accumulating the short pairs instead.

10.2 N cameras: occlusion and topology

A point enters the model only if at least two cameras see it. With two cameras, anything hidden from either one is lost; the reconstructed surface is only the band that is simultaneously front-facing and unobstructed for both — for objects with complex topology (handles, tunnels, deep concavities) that band can be a small fraction of the surface, and the holes it leaves are the familiar occlusion artifacts.

A useful back-of-envelope model: if a surface point is occluded in any single view independently with probability q , then

$$P(\text{reconstructible}) = P(\geq 2 \text{ of } N \text{ see it}) = 1 - q^N - N q^{N-1}(1 - q). \quad (15)$$

Independence is optimistic for clustered cameras (they get occluded together) and pessimistic for well-spread ones — spreading the cameras is what makes the assumption approachable.

Example 15 (occlusion coverage). Take $q = 0.3$ (a fairly convoluted scene). By (15): $N = 2$: $(1 - q)^2 = 0.49$ — half the surface. $N = 4$: $1 - 0.3^4 - 4 \cdot 0.3^3 \cdot 0.7 = 1 - 0.0081 - 0.0756 \approx 0.92$. $N = 6$: $1 - 0.3^6 - 6 \cdot 0.3^5 \cdot 0.7 \approx 0.99$. Going from two to four cameras nearly doubles the reconstructible surface; four to six mops up the remainder. The marginal camera always buys less — but for high-genus objects the last percent *is* the topology.

Topology sharpens the point. Silhouette-based reasoning can never recover concavities: the best any number of silhouettes gives is the visual hull [Lau94], which fills in bowls and blocks handles. Capturing the true genus of an object — the hole of a handle, the tunnel through an arch — requires cameras whose *sightlines pass through the hole*, seeing background where the hull would put material; photo-consistency frameworks (space carving [KS00], patch-based multiview stereo [FP10]) then remove exactly the voxels/patches contradicted by those views. So for complex topology, added cameras should be *spread to thread the object’s openings and look into its concavities*, not packed for baseline; the standard benchmark evidence [SCD+06] is that view coverage, more than per-view quality, is what separates complete reconstructions from holed ones. In this system’s terms: the two rig cameras provide the calibrated metric backbone, and subsection 11.1’s registered auxiliary photos are the cheap way to add exactly such coverage views.

Two practical costs scale with N : (i) calibration and synchronization — each camera needs its own [section 5](#) session against the same target, all cameras a common trigger; (ii) computation — pairwise geometry (fundamental matrices, rectifications) grows as $N(N-1)/2$, though in practice a hub-and-spoke set against one reference camera suffices for gating.

10.3 Resolution

For a fixed field of view, the focal length *in pixels* scales with the sensor’s linear resolution: doubling from 640×480 to 1280×960 doubles f from 800 to 1600 px. Two consequences follow directly.

Depth precision. Matching accuracy σ_d is an approximately constant *fraction of a pixel* (our block matcher’s parabolic refinement sits around 0.1–0.3 px on textured surfaces), so in [\(13\)](#) the denominator’s f doubles while σ_d stays put: σ_Z halves. Depth precision is linear in resolution — until lens blur, diffraction, or per-pixel photon noise (smaller pixels collect less light) starts inflating σ_d and returns diminish.

Lateral detail. The pixel footprint on the scene (ground sampling distance) is Z/f metres per pixel. Surface detail must span at least ~ 2 pixels to be matchable, so the smallest reconstructible geometric feature is about $2Z/f$ — this, not depth noise, is what limits how much of a complex object’s fine structure (thin ribs, small holes) survives into the model. In the fidelity sense of [subsection 10.2](#): resolution sets the smallest feature you can capture, view coverage sets whether you capture it at all.

Example 16 (resolution budget). Experimental rig, $Z = 4.7$ m: footprint $Z/f = 4.7/800 = 5.9$ mm/px, smallest matchable feature ≈ 12 mm, $\sigma_Z \approx 23$ mm ([Example 13](#)). Doubling both cameras’ resolution: footprint 2.9 mm/px, feature floor ≈ 6 mm, $\sigma_Z \approx 12$ mm. Everything improves by the same factor of two.

10.4 Unequal resolutions

Nothing in the model requires the cameras to match: each carries its own $\mathbf{K}$ (and distortion), the fundamental matrix [\(10\)](#) exists between any pair, and triangulation consumes pixels through each camera’s own $\mathbf{P}$. What changes is the error budget. Each camera contributes *angular* noise σ/f_i (radians); ray-intersection variance adds them, so a pair’s effective noise is

$$\sigma_\theta = \sqrt{(\sigma_1/f_1)^2 + (\sigma_2/f_2)^2} : \quad (16)$$

the *coarser camera dominates*, and money spent on one camera is mostly wasted unless the other keeps up.

Example 17 (upgrading one camera vs both). Rig of [Example 13](#), $\sigma = 0.3$ px per camera. Both at $f = 800$: $\sigma_\theta = \sqrt{2}(0.3/800)$. Upgrade *one* camera to $f = 1600$: $\sigma_\theta = (0.3/800)\sqrt{1 + \frac{1}{4}}$, an improvement of only $\sqrt{1.25/2} \approx 0.79$ — depth error drops from 23 mm to 18 mm. Upgrade *both*: factor 0.5, 12 mm ([Example 16](#)). Half the hardware upgrade buys about a fifth of the benefit.

Two implementation notes for mixed rigs. First, rectification ([section 8](#)) forces a *shared* $\mathbf{K}_{\text{new}}$; our implementation averages the two intrinsic matrices, which for very unequal cameras

oversamples the coarse image (interpolated pixels carry no new information) — resampling both to the coarser camera’s grid keeps the cost of dense matching honest, at the price of discarding the fine camera’s lateral detail. Second, the DLT (11) as implemented weights all rows equally; with strongly unequal cameras the rows of the finer camera deserve proportionally more weight ($\propto f_i/\sigma_i$ after each view’s normalization), a small planned refinement alongside the Hartley–Sturm step (section 16).

11 Planned extensions

11.1 Auxiliary photo collections

Still photographs of the same volume — arbitrary viewpoints, possibly a different camera — densify coverage and fill occlusions. The integration path follows the structure-from-motion recipe of Photo Tourism [SSS06]: detect scale-invariant features [Low04] in the photos *and* in the two calibrated video frames; match with a RANSAC gate [FB81]; register each photo to the rig’s world frame by resection from matches against already-triangulated points; then re-triangulate all tracks with the registered views via the existing N -view DLT (section 7); finally refine everything with bundle adjustment [TMHF00]. The two calibrated cameras anchor the metric scale — the classic scale ambiguity of pure SfM does not arise. Dense multiview stereo over the registered set (e.g. patch-based methods [FP10]) is a further optional stage.

11.2 Range-finder fusion (optional, kept open)

The system is designed to work without a range finder; this subsection records what a single-beam laser ranger would add if one arrives later, so the option stays open at zero present cost. Two integration modes: (i) if the range finder is rigidly mounted and extrinsically calibrated to the rig, each range sample is simply another world-frame point with small σ_Z , fused into the cloud with inverse-variance weights; (ii) if it produces an independent scan, the scan is registered to the vision cloud by closed-form absolute orientation [Hor87] initialized correspondence-free and refined with ICP [BM92].

At calibration time (with the rotating display of subsection 5.6). Four distinct improvements. (i) *Independent absolute scale*: the vision-only metric chain hangs entirely on the display’s pixel pitch; a ~ 2 mm-accurate range at 4.7m is a 0.04% scale reference — a second chain that turns a silent global scale bias into a detectable discrepancy. (ii) *Orbit metrology*: aimed so the rotating screen sweeps through the beam, the periodic range trace $d(t)$ yields ω , phase, orbit radius and axis distance at millimetre level, independently of every camera, and ties laser and camera timestamps to the same physical event. (iii) *The focal–depth weak direction* (subsection 5.4) is attacked head-on: an absolute distance to the target plane is exactly the constraint type planar homographies lack. (iv) *The plate calibrates the laser*: if the spot is visible to the cameras, the sweeping plate carries it through continuously varying depths; each frame gives (range, pixel₁, pixel₂), the cameras triangulate the spot, and the beam’s ray in the rig frame (~ 5 DOF) falls out of a least-squares line fit — no manual procedure at all.

At run time. Laser error is roughly constant while stereo error grows as Z^2 (13), so the laser anchors the far field and the absolute scale (sparse metric skeleton, dense stereo flesh); it provides depth in texture deserts where block matching has nothing to match, and used as a

prior it narrows the disparity search there; and a persistent bias between laser and stereo depth at the same spot is an online health monitor for the metric chain, complementing the **F**-drift monitor of [section 6](#).

Example 18 (laser–stereo crossover). With $\sigma_{\text{laser}} = 2$ mm constant and the rig’s $\sigma_Z = Z^2\sigma_d/(fB)$: equality at $Z = \sqrt{0.002 \cdot 800 \cdot 0.5/0.42} \approx 1.4$ m. Nearer than that, stereo wins per point; beyond it the laser is better along its ray — tenfold better at 4.7 m (2 mm vs 23 mm) — while stereo retains the advantage of density. Inverse-variance fusion realizes both.

12 Numerical implementation notes

All decompositions reduce to one kernel: a one-sided (Hestenes) Jacobi SVD [\[GV13\]](#) for $m \geq n$ matrices, $n \leq 9$ in every present use (the design matrices of [\(5\)](#), [\(6\)](#), [\(11\)](#), and the 8-point system). Jacobi SVD was chosen over Golub–Kahan bidiagonalization for its simplicity (~ 100 lines, no workspace queries), unconditional convergence, and excellent relative accuracy on small matrices; its $O(mn^2)$ -per-sweep cost is irrelevant at these sizes. Homogeneous solutions are the right singular vector of the smallest singular value; rank-2 enforcement for **F** zeroes the smallest singular value and recomposes.

Conditioning is handled where it matters: design matrices built from raw pixel coordinates mix terms of order 1, u , and u^2 ($u \sim 10^2$ pixels), giving condition numbers around 10^8 ; Hartley normalization [\[Har97\]](#) (translate to centroid, scale to RMS radius $\sqrt{2}$) reduces this to $O(10)$ and is applied in both the 8-point and the homography estimators, with results denormalized afterwards. All fundamental matrices are canonicalized (unit Frobenius norm, largest-magnitude entry positive) so that analytic and estimated matrices are directly comparable — this is what makes the drift monitor of [section 6](#) a plain matrix norm. Calibration runs Zhang’s closed form inside the small alternation loop of [subsection 5.3](#); a full Levenberg–Marquardt refinement over all parameters is the planned upgrade ([section 16](#)).

The code is strict C99 (`-std=c99 -pedantic -Wall -Wextra` clean), depends only on `libm`, allocates only in unavoidable places (design matrices, images, clouds), and contains no hidden state except the demos’ seeded generators. Determinism is treated as a feature: identical inputs give bit-identical outputs across runs, so every regression is reproducible.

13 Basic results

Three validation layers accompany the library. `tests/test_mv.c` checks exact properties on noiseless data (30 assertions: SVD reconstruction and orthonormality, ray/projection consistency, [\(9\)](#) to 10^{-8} , exact homography and triangulation recovery, exact intrinsic/pose recovery by Zhang’s method, Gray-code encode/decode round-trip, row alignment after rectification); all pass. The two demo programs are controlled noisy experiments.

Two-camera reconstruction (`demo/synthetic.c`). Two cameras with $f = 800$ px, 640×480 images, baseline $B = 0.5$ m, the second camera yawed 6° inward, observe 250 points drawn uniformly in a 2×1.5 m window at depths 3.5–6 m; projections carry Gaussian noise $\sigma = 0.3$ px per coordinate (seed fixed).

The numbers agree with theory: the epipolar residual ≈ 0.34 px matches the injected noise, and the RMS 3D error of 2.65 cm matches the 2.3 cm depth-noise floor computed by hand in

Quantity	Value
points visible in both views	250/250
$\ \mathbf{F}_{\text{analytic}} - \mathbf{F}_{\text{8-point}}\ _F$ (unit-norm)	4.5×10^{-3}
mean symmetric epipolar distance (analytic $\mathbf{F}$)	0.345 px
mean symmetric epipolar distance (8-point $\mathbf{F}$)	0.341 px
triangulation RMS 3D error	0.0265 m
triangulation max 3D error	0.0876 m
RMS reprojection error	0.218 px
mean row misalignment $ v_1 - v_2 $ before rectification	0.764 px
mean row misalignment after rectification	0.0 px

Table 2: Reconstruction experiment, seed 42 (reproduce with `make demo`).

Example 13 (the measured value also contains the smaller lateral components). The implementation performs at the noise floor of the geometry rather than being implementation-limited. The reconstructed cloud is written to `out_cloud.ply`.

Calibration (`demo/calibrate.c`). A camera with ground-truth $f_x=800$, $f_y=805$, $(c_x, c_y)=(322, 238)$, zero skew and zero distortion photographs the letter-page target (subsection 5.4; 7×9 corners, 24 mm squares) in six poses with tilts up to 35° at 0.45–0.65 m; corners carry $\sigma = 0.3$ px noise.

Quantity	Value
f_x error	−7.0 px (0.9%)
f_y error	−7.9 px (1.0%)
c_x, c_y error	−1.0, −1.8 px
mean rotation error over views	0.31°
mean translation error over views	5.2 mm
reprojection RMS	0.48 px

Table 3: Calibration experiment, seed 7, six views, $\sigma = 0.3$ px (reproduce with `make demo`).

Sub-1% intrinsics and millimetre-level poses from six noisy views of one printed page is the expected performance of Zhang’s method without nonlinear refinement; near-frontal pose sets degrade the focal estimate markedly (weak observability, subsection 5.4), which the experiment reproduces if the tilt angles are reduced. The fitted (k_1, k_2) absorb a little noise as a near-zero distortion *curve* while being individually large — the identifiability effect discussed in subsection 5.3. A Levenberg–Marquardt refinement stage is the roadmap item that tightens all of these numbers.

Tracking a moving object (`demo/track_robot.c`). A disk-shaped vacuum robot (radius 0.17 m, height 0.09 m) wanders the monitored volume for 20 s at 0.3 m/s with random turns, on a floor 0.8 m below the standard rig. Feature points sit on its cylindrical bumper band; per camera, the far side of the band is self-occluded (surface-normal test), so each frame observes only a partial arc — on average 16.7 of 36 band points survive visibility, image bounds, and the 1.5 px epipolar gate built from the analytic $\mathbf{F}$ (10). Each frame’s surviving correspondences are

triangulated and the ground-plane circle of *known* radius is fit to them by Gauss–Newton; the fitted center is scored against the simulated trajectory.

Quantity	Value
frames / rate	600 / 30 fps
mean visible band points	16.7 of 36
center RMS error, lateral (x)	5.1 mm
center RMS error, depth (z)	7.7 mm
center RMS error, horizontal	9.2 mm
center max error	26.2 mm
naive-centroid RMS (contrast)	116.5 mm
speed RMS error (± 2 -frame difference)	95 mm/s

Table 4: Robot-tracking experiment, seed 12345, $\sigma = 0.3$ px (reproduce with `make demo`). Trajectories written to `out_track.ply`.

Three lessons, each checkable by hand. First, *model knowledge removes the partial-view bias*: the centroid of a half-arc of a circle of radius R sits $2R/\pi \approx 108$ mm from the center — the naive centroid’s measured 116 mm RMS is exactly this geometric bias, and the known-radius fit eliminates it (9 mm). Second, the per-frame precision follows the error laws: single-point depth noise $\sigma_Z \approx 23$ mm (Example 13) averaged over ~ 17 points predicts $\sigma \approx 5.6$ mm, and the measured 7.7 mm depth RMS carries the expected extra factor from fitting a center to a partial arc; the residual anisotropy (z worse than x) is the depth law itself. Third, *differentiation amplifies noise*: 9.2 mm of position noise differenced over 4/30 s predicts $\sqrt{2} \cdot 9.2/0.133 \approx 98$ mm/s of speed noise — the measured 95 mm/s, i.e. a third of the robot’s actual speed, which is why velocity estimates in practice come from a motion-model filter (constant velocity Kalman or similar) rather than raw differencing; such a filter is a natural later addition to the temporal-statistics roadmap item. This experiment is geometry-level simulation (points, not pixels); the rendered-image tier — the same scenario in a physically-based renderer exercising the dense matcher — is the planned next validation stage.

13.1 Error budget and consistency: is there a bug?

Measured errors alone cannot distinguish “the physics” from “a defect.” `demo/diagnose.c` therefore runs three targeted checks, built on the principle that noise-driven error and bias behave differently: a true bias survives $\sigma \rightarrow 0$, and honest noise propagation is *linear* in σ in the operating regime. All numbers below are deterministic and reproducible.

A. Triangulation vs the depth law. The per-point prediction $\sigma_Z = Z^2 \sqrt{2} \sigma / (fB)$ (13), evaluated on the same sampled points, against the measured depth RMS:

Agreement with the analytic law to 1% at every noise level, exact zero at $\sigma = 0$, and both error components scaling exactly linearly. The DLT’s algebraic (rather than reprojection-optimal [HS97]) cost is invisible at these noise levels — the 1% residual bounds it.

B. Tracking estimator across noise levels. The same sweep for the robot experiment, with the *arc factor* = measured center-depth RMS over the naive $\sqrt{n}$ -averaged per-point prediction:

σ (px)	measured z RMS	predicted	ratio	lateral RMS
0.0	0.000 mm	0.000 mm	—	0.000 mm
0.1	8.75 mm	8.66 mm	1.01	1.37 mm
0.3	26.2 mm	26.0 mm	1.01	4.12 mm
0.6	52.4 mm	52.0 mm	1.01	8.24 mm

Table 5: Triangulation noise sweep (250 points, seed 42).

σ (px)	center x RMS	center z RMS	arc factor
0.0	0.000 mm	0.000 mm	—
0.1	1.72 mm	2.39 mm	1.13
0.3	5.12 mm	7.68 mm	1.21
0.6	10.7 mm	40.0 mm	3.14

Table 6: Tracking noise sweep (600 frames, seed 12345).

Zero bias at $\sigma = 0$ (the fit recovers exact circles to machine precision), near-linear scaling from 0.1 to 0.3 px with a mild, stable arc factor of 1.1–1.2 — and then a genuine *nonlinearity* at $\sigma = 0.6$: depth error jumps to 40 mm (factor 3.1) instead of the linear 15 mm. This is not a defect but a measured limitation of the estimator: per-point depth noise at $\sigma = 0.6$ reaches ~ 46 mm — a quarter of the robot’s *radius* — and the circle fit assumes isotropic residuals while the actual noise is strongly depth-elongated, so the fitted center gets dragged along z . The fix, when needed, is a Mahalanobis-weighted fit using the known per-point error ellipse (same family as the weighted DLT of [subsection 10.4](#)). At the operating point $\sigma = 0.3$ px the estimator sits comfortably in its linear regime.

C. Calibration: is -7 px a typical draw? The single reported focal error ([Table 3](#)) is one noise realization. Across ten seeds at $\sigma = 0.3$ px the f_x errors are $\{-4.0, -1.0, +3.5, +1.5, -2.5, +3.4, -7.0, +0.7, +\dots\}$ mean -0.08 px (*unbiased*), sample standard deviation 3.5 px. The demo’s -6.95 px is a 2σ draw — unlucky, not anomalous. At $\sigma = 0.15$ px the mean $|f_x|$ error is 1.41 px, exactly half the 2.82 px of $\sigma = 0.3$: linear again.

Verdict. Every check a bias-type bug could fail is passed: all estimators are exact on noiseless data (here and in the 30-assertion unit suite), all errors scale linearly with input noise in the operating regime, the dominant error channel matches its analytic prediction to 1%, and the calibration estimator is unbiased in expectation. The error sources, ranked: (1) pixel noise through the depth law — dominant, modeled, verified; (2) estimator geometry factors (partial-arc factor 1.2; calibration’s finite-view variance of 3.5 px std, inflated further by weak pose diversity, [subsection 5.4](#)); (3) unmodeled noise *anisotropy* in the circle fit — negligible at 0.3 px, dominant by 0.6 px; (4) DLT algebraic suboptimality and the 3.5σ gating truncation — both bounded below the 1% level here. A bug can never be excluded, but everything measurable currently behaves exactly as the theory of [section 7–section 8](#) says it must.

13.2 Statistical decision rules: when the numbers would say “bug”

The preceding subsection can be formalized into a standing test battery. Under the null hypothesis H_0 = “implementation correct,” the measurement model (i.i.d. Gaussian pixel noise of scale σ) fixes the sampling distribution of every summary statistic we compute; a bug is declared when a statistic leaves its null band. Because the whole system is deterministic, any flag is exactly reproducible — a fired tripwire can be replayed under a debugger.

- T1 — exactness at $\sigma = 0$.** Under H_0 every estimator is algebraically exact, so noiseless residuals are bounded by floating-point round-off amplified by conditioning, $\varepsilon \cdot \kappa \lesssim 10^{-9}$. This test has *no* statistical tolerance: any exceedance is a defect (sign error, wrong index, forgotten term). It is the single most powerful test in the suite and is what the 30-assertion unit suite automates.
- T2 — prediction-ratio test.** For a measured RMS over N residuals against an analytic prediction, the sample RMS under H_0 fluctuates with relative standard deviation $\approx 1/\sqrt{2N}$ (χ^2). Flag when $|r - 1| > 3/\sqrt{2N}$. A ratio that is stable but $\neq 1$ across σ and seeds is the signature of a wrong constant (missing $\sqrt{2}$, wrong baseline, mis-scaled f).
- T3 — scaling-exponent test.** Regress $\log e$ on $\log \sigma$: under H_0 , slope = 1 in the linear regime. Slope ≈ 2 betrays a missing square root or an estimator leaving its regime; slope $\rightarrow 0$ at small σ betrays an additive floor — fitting $e(\sigma) = \sqrt{b^2 + a^2\sigma^2}$ and finding b above the numerical floor reveals a hidden bias even when T1 cannot be run.
- T4 — ensemble bias test.** Over M independent noise realizations, the standardized mean $|\bar{e}|/(s/\sqrt{M})$ is Student- t under H_0 ; flag beyond 3. Distinguishes systematic bias from an unlucky draw.
- T5 — residual-consistency (χ^2) test.** A least-squares fit with p parameters against n residual coordinates must leave $\text{RMS} = \sqrt{2}\sigma\sqrt{1 - p/n}$ (per-point distance); the ratio fluctuates with relative std $1/\sqrt{2(n - p)}$. A ratio *above* the band means the model or estimator is not absorbing what the noise model says it should (underfit, suboptimal estimator, contamination); *below* means overfit or a too-permissive model.
- T6 — outlier-count test.** The epipolar gate’s rejection count among true correspondences is Poisson with rate given by the noise tail beyond the gate; a large excess indicates contamination or a mismatched noise model, a large deficit an accidentally widened gate.

The one standing flag. T5 fires on calibration: across ten seeds the fitted reprojection RMS averages 0.484 px against the noise-model prediction $\sqrt{2}\sigma\sqrt{1 - 42/756} = 0.412$ px — ratio 1.17, roughly twenty standard errors outside the null band. The statistics are working exactly as intended: they have detected that the alternation estimator of [subsection 5.3](#) (Zhang closed form + linear radial step) is *not* the joint maximum-likelihood fit — it minimizes algebraic surrogates, leaving $\sim 17\%$ systematic residual above the noise floor. This is the documented, expected cost of the missing Levenberg–Marquardt stage, and it hands that roadmap item a quantitative acceptance criterion: *after LM refinement, the T5 ratio must fall to 1.00 ± 0.03* . If it does not, *then* we have a bug. This is the general pattern the project commits to: every claimed improvement must move a named statistic into a predicted band, and every statistic outside its band must be either explained to this standard or treated as a defect.

Test	Bug-flag condition	Current value	Verdict
T1 exactness ($\sigma=0$)	any residual $> 10^{-9}$	$\leq 10^{-12}$	pass
T2 ratio, triangulation	$ r - 1 > 0.13$ ($N=250$)	$r = 1.01$	pass
T3 slope, triangulation	$ \text{slope} - 1 > 0.1$	1.00	pass
T3 slope, tracking 0.1 $\rightarrow$ 0.3	$ \text{slope} - 1 > 0.1$	1.06	pass
T3 slope, tracking 0.3 $\rightarrow$ 0.6	—	2.4	fired ^a
T4 bias, calibration f_x	$ t > 3$ ($M=10$)	$t = 0.07$	pass
T5 residual, triangulation	$ r - 1 > 0.13$	$r = 1.03$	pass
T5 residual, calibration	$ r - 1 > 0.08$	$r = 1.17$	fired^b
T6 gate rejections	excess over noise tail	$6 / \sim 10^4$	pass

Table 7: Tripwire standings. ^aInvestigated and resolved: the known-radius circle fit leaves its linear regime at $\sigma = 0.6$ px (noise anisotropy, Table 6); a regime boundary, not a defect. ^bGenuine standing flag, see text.

14 Scenario studies

Four concrete monitoring scenarios, each simulated end-to-end against ground truth in the deterministic demo style (all at the geometry tier: points, not pixels — detections are assumed, estimation is exercised; each result is therefore an upper bound whose image-tier feasibility is noted). Together they instantiate the mover-separation architecture of subsection 9.3.

14.1 Insect counting and entry/exit statistics

Point-like targets enter the monitored box through a random face, fly an Ornstein–Uhlenbeck random walk, and leave (`demo/track_insects.c`: 300 s, spawn rate 0.1/s, joint detection probability 0.9 per frame, *plus two uniform clutter detections per camera per frame*). Stereo association is by the epipolar gate alone; a nearest-neighbor tracker with 3-hit confirmation and 5-miss termination maintains identities; confirmed births/deaths map to box faces. Results (seed 777): ground truth 31 insects, **29 tracks confirmed** (two lived too briefly to confirm); entry/exit face histograms match ground truth to within ± 2 per face; track position RMS 24.4 mm — the single-point depth law of Example 13, as an unmodeled point target admits no $\sqrt{n}$ averaging. Of $\sim 36,000$ clutter detections, only 231 pairs slipped through the epipolar gate and *none* survived track confirmation: geometric gating plus temporal confirmation suppresses clutter completely at these densities. Image-tier caveat: at 4.7 m a 5 mm insect subtends 0.85 px — below the 2-px feature floor of subsection 10.3 — so real detection must be temporal-differencing of sub-pixel blobs; counting accuracy will be detection-limited, not geometry-limited.

14.2 Diurnal cycle and artificial lighting

The cameras double as calibrated photometers (Appendix; radiometric calibration assumed): mean scene luminance is one number per frame. `demo/lightlog.c` simulates 120 days at 1-minute resolution — natural light with drifting sunrise (-1.5 min/day), per-day weather (0.4 – $1.0\times$), slow AR(1) cloud noise, two lamps on noisy schedules, two rare middle-of-the-night events — and analyzes the sum. Step detection (median-filter differencing): **484/484 on/off events found, 0 missed, 33 false alarms** (cloud edges), timing RMS 1.02 min, lamp identity by step magnitude **484/484 correct**, including both night events. The natural component is recovered by subtracting the reconstructed lamp state, with a *nightly re-anchor* (pre-dawn darkness

forces the natural component to zero — a physical constraint that stops false-alarm error from propagating across days). Sunrise is then estimated per day by two-level template inversion (crossings of 15% and 35% of the day’s robust peak invert the known diurnal shape analytically; amplitude and shape cancel): offset 0.7 min (unbiased), day-to-day jitter 25.3 min RMS — the genuine information floor imposed by 100-min-correlated cloud cover, not an estimator defect — and the seasonal drift is recovered as

$$\widehat{\text{drift}} = -1.61 \text{ min/day} \quad (\text{true} = -1.50, \sigma_{\text{slope}} \approx 0.07).$$

A 30-day run is statistically *underpowered* for this drift ($\sigma_{\text{slope}} \approx 0.7$): a season of data is what makes the trend measurable — the T4 logic of [subsection 13.2](#) applied to experiment design.

14.3 Human occupancy

A known three-person cohort (heights 1.62/1.75/1.88 m) makes 13 scheduled visits over a simulated day (`demo/track_people.c`): enter at the door, walk to desk or window, linger, exit. Each person is a cylinder with self-occluding torso-band features plus a head point; per frame the known-radius circle fit gives ground position and the triangulated head gives height. Results (seed 31337): **13/13 episodes detected, 13/13 identities correct** (nearest cohort height; the 130 mm height gaps dwarf the 0.3 mm height RMS), 13/13 trajectory targets correct, ground-position RMS 14.1 mm, entry latency 0.2 s (the 3-frame confirmation), per-person occupancy minutes recovered to 0.1 min. Caveats: identity-by-height is valid only for small cohorts with distinct heights (it is a household biometric, not a general one), and the idealized head/torso detections make these upper bounds — real image-tier person detection is where the learned-detector boundary of [subsection 9.1](#) sits.

14.4 What the scenarios share

All four reuse the same small kernel — epipolar gating, triangulation, model-constrained fitting, temporal confirmation — plus one scenario layer each (multi-target association; photometric step decomposition; identity classification). The common upgrades they all point to: constant-velocity Kalman filtering (speed estimates, [Table 4](#)), the Mahalanobis-weighted fits of [subsection 13.1](#), and the background-subtraction front end of [subsection 9.3](#).

15 Person detection and face recognition with open-source components

The learned-detector boundary drawn in [subsection 9.1](#) does not have to remain outside the system: mature open-source detectors and face recognizers can be attached *without* touching the dependency-free core, and the occupancy scenario ([subsection 14.3](#)) already defines the exact seam they plug into. This section records the component landscape, the integration architecture, and — the question that determines what to promise — how accuracy differs between a pre-registered cohort and unknown faces.

15.1 Component landscape

Three roles, each with solid open-source fillings: *person detection* — YOLO-family single-shot detectors [[RF18](#)] (the original darknet implementation is plain C; Apache-licensed successors

such as YOLOX run everywhere), or classical HOG pedestrian detection in OpenCV as a weak-but-tiny fallback; *face detection + landmarks* — RetinaFace [DGV⁺20] or its lightweight successors (SCRFD, YuNet), which return a box plus five landmarks (eyes, nose, mouth corners); *face embedding* — deep metric networks mapping an aligned 112×112 crop to a unit vector such that same-person pairs are close: FaceNet established the recipe [SKP15], ArcFace-class models are the current open-source standard [DGXZ19]. The practical integration seam for all three is an ONNX-exported model behind the ONNX Runtime C API (MIT-licensed, single shared library, plain-C interface) — the one boundary object between the C99 world and the learned world. License note: several popular YOLO packagings are AGPL; the Apache/MIT alternatives keep the system’s licensing simple.

15.2 Piping it in

The architecture is a *sidecar oracle*: the core stays pure C99 and treats the detector exactly as the scenario demos treat their simulated detections. Per frame and per camera, the sidecar consumes the image and emits rows of `(frame, camera, class, box, landmarks, embedding)` — a text/CSV stream over a pipe or file, trivially parseable, recordable, and replayable (determinism is preserved by recording the sidecar’s output once and replaying it in tests — the oracle’s answers become fixtures). The geometric core then does what it already does:

1. *stereo association*: detections in the two cameras are paired by the epipolar gate on box centers/landmarks (section 6) — the same gate that suppressed clutter in subsection 14.1;
2. *triangulation*: paired boxes give the person’s 3D position; paired eye landmarks give a metric face position and, with the floor plane, the height biometric of subsection 14.3;
3. *track-level identity*: embeddings are not judged per frame but accumulated per *track* (quality-weighted average of embeddings across the track’s frames): the tracker turns many marginal glimpses into one confident identity, and the identity then travels with the track through frames where the face is turned away entirely.

Identity fusion is a small score-level combination: face-embedding similarity where a face is visible, the height/gait geometric cues otherwise — the heterogeneous-evidence pattern of the appendix (subsection A.7) applied to identity.

15.3 Pre-registered vs unknown faces

The accuracy question splits into three regimes with *very* different numbers.

Verification (1:1) and closed-set identification (1: N , person guaranteed enrolled). On the classic verification benchmark LFW [HRBLM07] modern open-source models exceed 99.5% — the benchmark is saturated and mostly measures that nothing is broken. The more honest unconstrained figure is IJB-C [MAD⁺18] verification at strict false-accept rates: ArcFace-class models reach a true-accept rate around 96–97% at $\text{FAR} = 10^{-4}$ [DGXZ19]. For a *pre-registered household cohort* — N of order 10, controlled enrollment photos, closed-set — rank-1 identification is effectively $\geq 99\%$ whenever the probe image is adequate: with guaranteed gallery membership, the decision is an argmax among a few well-separated identities, and errors are driven almost entirely by probe quality, not confusion.

Open-set identification (the person may be unknown). This is a different problem: the system must first decide *whether* the probe is anyone enrolled, which requires thresholding an absolute similarity score rather than taking an argmax. Impostor scores from unknown faces overlap the tail of genuine scores, so every operating point trades false alarms (visitor labeled as a resident) against misses (resident labeled unknown). NIST’s identification evaluations [GNH19] quantify the cost: even leading algorithms give up substantial accuracy relative to closed-set — at a false-positive identification rate of 1%, miss rates range from a few percent with high-quality frontal probes to *tens of percent* with surveillance-quality probes, and degradation grows with gallery size and probe-gallery quality gap. The rule of thumb this system should adopt: **closed-set household identification is a solved problem at our scale; reliable “this is a stranger” detection is not free** — it needs a margin-calibrated threshold, benefits enormously from track-level embedding fusion, and should be reported with its false-alarm/miss operating point, never as a single accuracy number.

The resolution gate. All of the above presumes enough face pixels, and here our own [subsection 10.3](#) bites hard.

Example 19 (face pixels at range). A face is ≈ 0.20 m tall with 0.064 m interocular distance. At the experimental rig’s $f = 800$ px and $Z = 4.7$ m: face height $0.20 \cdot 800 / 4.7 \approx 34$ px, interocular ≈ 11 px — far below the $\gtrsim 80$ –112 px face crops the embedding networks are built for; expect severely degraded embeddings. For a 100 px face at 4.7 m the rig needs $f = 100 \cdot 4.7 / 0.20 = 2350$ px — a 4K-class sensor, or a normal sensor with a narrow-field lens.

The systems answer follows the appendix’s blending argument ([subsection A.7](#)): do not demand identity everywhere. Add one *identity camera* — a narrow-field unit aimed at the natural chokepoint (the door of [subsection 14.3](#), where every track begins and ends) — capture the face at high resolution once per entry, and let the tracker carry that identity through the wide-angle geometric volume. Identity becomes an attribute assigned at a favorable moment and *propagated by geometry*, rather than a per-frame recognition burden. A final scope note befitting the controlled environment: the enrollment gallery is the household’s own, processing is local, and the open-set threshold should be set to prefer “unknown” over misattribution.

15.4 Plugin matrix: OSS solutions, features, and license isolation

[Table 8](#) surveys the practical candidates. Feature columns: person detection, face detection, landmarks, embeddings, ONNX exportability (i.e. runs behind the single ONNX Runtime seam), and CPU-only viability. The last column states the *required* degree of separation to avoid license contamination of the MIT-clean core:

- **link** — permissive license (MIT/BSD/Apache/BSL): may be linked into the same process; no obligations beyond attribution.
- **process** — AGPL: never link or import; run as a separate OS process communicating *data only* (frames out, text rows back). The combined system must not be distributed as one work, and if the sidecar is ever offered as a network service, AGPL source obligations attach to the sidecar alone.
- **weights apart** — code is permissive but the pretrained *weights* carry a research/non-commercial license: treat weights as user-fetched artifacts at install time, never redistributed with the system.

Architecturally we put *every* detector behind the sidecar protocol of [subsection 15.2](#) regardless of license — hygiene first — so the license question collapses to “which side of the pipe, and who ships the bytes.”

Solution	License (code/weights)	Person	Face	Lmk	Embed	ONNX	CPU	Isolation
darknet YOLOv3/v4	MIT-like / same	✓	—	—	—	part	slow	link
YOLOX	Apache / Apache	✓	—	—	—	✓	ok	link
Ultralytics v8/v11	AGPL / AGPL ^a	✓	—	pose	—	✓ ^a	ok	process
OpenCV (HOG/YuNet/SFace)	Apache / Apache	weak	✓	5pt	✓	n/a	✓	link
MediaPipe	Apache / Apache	pose	✓	✓	—	—	✓	link
dlib / face_recognition	BSL / public	HOG	✓	68pt	✓	—	slow	link
InsightFace (SCRFD+ArcFace)	MIT / research ^b	—	✓	5pt	✓	✓	✓	weights apart
facenet-pytorch (MTCNN+FaceNet)	MIT / MIT	—	✓	5pt	✓	✓	ok	process ^c
ONNX Runtime (the seam)	MIT	— inference runtime for all ONNX rows —						link

Table 8: OSS plugin candidates. ^aAGPL applies to exported/converted weights too — conversion does not launder the license; mandatory process isolation, data-only IPC, no combined distribution. ^bInsightFace *code* is MIT but the model-zoo weights are research/non-commercial: fine in this controlled personal deployment, user-fetched at install, never redistributed. ^cLicense permits linking; the Python runtime makes a separate process the practical choice anyway.

The current default stack, license-clean end to end: YOLOX (person) and YuNet or SCRFD (face) and SFace or user-fetched ArcFace (embedding), all as ONNX graphs behind ONNX Runtime; the Ultralytics row exists because its tooling is convenient for experiments — inside its own fence.

15.5 Deployment architecture: hardware boundaries and communication

The communication patterns are deliberately boring: hardware trigger pulses for simultaneity ([subsection A.2](#)); USB3 raw frames into the metric host; a one-directional frame stream to the sidecar (shared memory locally, TCP if the sidecar moves to its own machine); text rows back. Nothing crosses the license boundary except data, every byte crossing it is recordable — which is exactly what makes the sidecar replayable in tests ([subsection 15.2](#)) — and the C99 core never links against anything but `libm`.

16 Roadmap

In dependency order: (1) saddle-point checkerboard corner detection with sub-pixel refinement, closing the printed-target path end-to-end (the display path of [subsection 5.5](#) is already detector-free); (2) the rotating-display fixture of [subsection 5.6](#): single-shot self-identifying pattern generation (M-array + Gray-coded frame counter) and the joint orbit fit; (3) Levenberg–Marquardt refinement of calibration (and later bundle adjustment [[TMHF00](#)]); (4) feature detection/description and robust matching (enables live correspondences, RANSAC-gated by [section 6](#)); (5) image warping by the rectifying homographies with bilinear resampling, closing the dense video path end-to-end; (6) temporal cloud statistics for static/moving segmentation ([section 9](#)); (7) photo-collection registration ([subsection 11.1](#)); (8) Hartley–Sturm optimal triangulation; (9) meshing; (10) the detector/identity sidecar of [section 15](#) (ONNX Runtime seam, recorded-oracle replay tests); and, if the hardware materializes, the optional range-finder integration of [subsection 11.2](#). Each step lands as a new module behind the same conventions ([Table 1](#)) with its own noiseless-exact tests and a synthetic experiment extending [section 13](#).

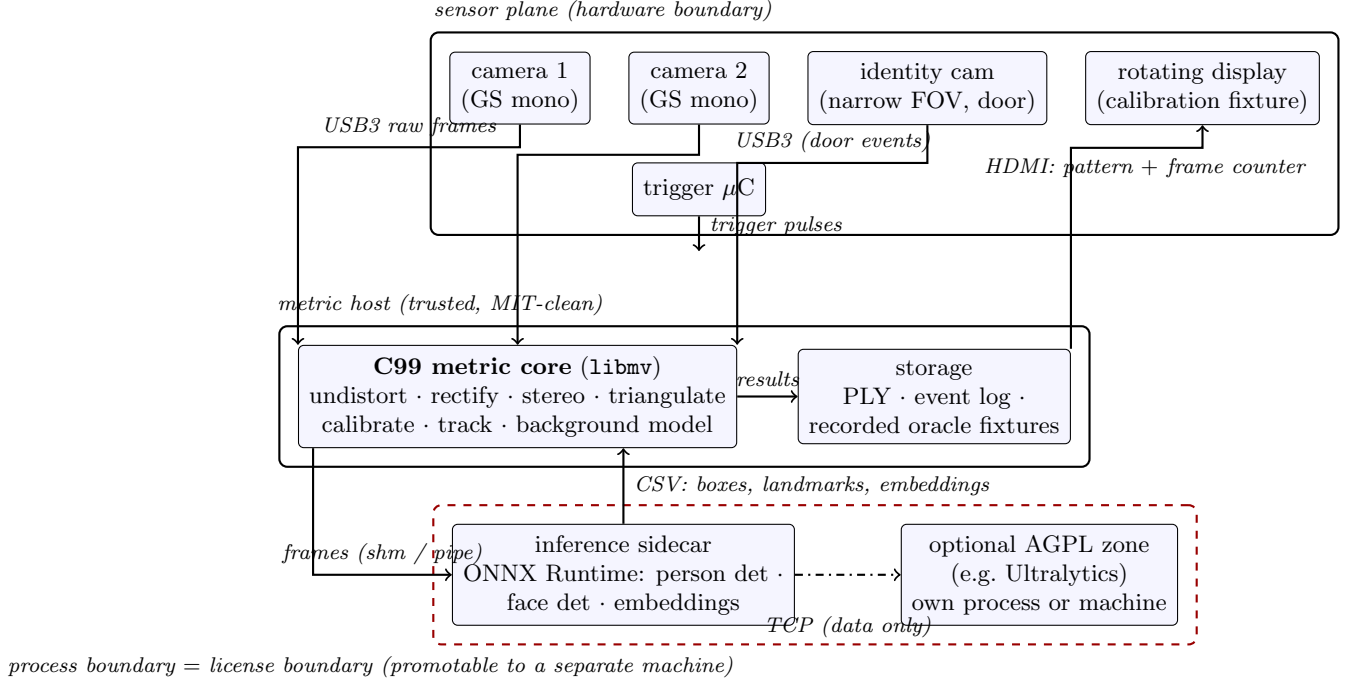


Figure 2: Deployment architecture. Solid boxes are hardware boundaries; the dashed red box is the license/process boundary — all learned components live behind it, exchanging only data (frames one way, text rows back), so it can be promoted from “separate process” to “separate machine” (a small GPU/NPU box) without any protocol change. The trigger microcontroller synchronizes both cameras, the identity camera, and — during calibration — the rotating display’s frame counter.

A Camera hardware and the quality of data

The capture hardware determines which error terms enter the pipeline before any algorithm runs. The organizing observation: cheap capture hardware mostly adds *biases* — rolling shear, synchronization parallax, compression structure, unmodelable distortion, focus creep — rather than noise. Noise we model and budget ([Example 13](#)); biases survive averaging, masquerade as geometry, and poison calibration silently. The specifications below are therefore ranked by which biases they admit.

A.1 Shutter: global vs rolling

A global-shutter sensor exposes all pixels over one common interval — each frame is a true snapshot, described by a single $\mathbf{P}$. A rolling-shutter sensor exposes and reads row by row: the bottom row of a frame is captured 10–30 ms after the top row. For a static scene the difference vanishes; for anything moving, each row samples a different instant and no single projection matrix describes the frame — the camera model of [section 4](#) itself breaks.

Example 20 (rolling-shutter shear on the rotating target). The rotating display of [subsection 5.6](#) moves at ≈ 27 px/s in the image. A rolling readout of 20 ms samples the top and bottom of the target 20 ms apart: a shear of $27 \cdot 0.02 \approx 0.5$ px across the corner grid — larger

than the 0.3 px noise budget, and *systematic*: it does not average away over frames but feeds a consistently wrong homography into calibration.

A.2 Synchronization: hardware trigger vs free-running

A trigger line starts exposure on all cameras from one electrical pulse (coincidence to microseconds). Free-running cameras can only be matched by software timestamps — a few *milliseconds* of jitter and drift. Triangulation assumes both cameras saw the point at the same instant; a time offset Δt on a point moving at speed v creates a phantom disparity $f v \Delta t / Z$ that is mathematically indistinguishable from depth.

Example 21 (the price of software synchronization). An object crossing the experimental rig at 1 m/s with 5 ms software sync: phantom disparity = $800 \cdot (1 \cdot 0.005) / 4.7 \approx 0.85$ px, which the depth law (13) converts to ≈ 47 mm of depth error — exceeding the rig’s entire 23 mm error budget at that range. For dynamic scenes, synchronization error dominates every other term; a trigger wire outperforms any camera upgrade. The trigger also permits phase-locking exposure against mains flicker and locking capture to the display’s frame counter (subsection 5.6).

A.3 Interface: UVC vs machine-vision stacks

UVC (USB Video Class) is the “any webcam just works” protocol (v412 on Linux, no vendor SDK). Its convenience reflects its purpose — streaming video to humans: at higher resolutions frames usually arrive *MJPEG-compressed*, and JPEG’s 8×8 block artifacts are structured noise at exactly the spatial frequencies where sub-pixel corners and block matching live — a *correlated* contribution to σ_d that does not average like photon noise. UVC also offers only coarse loosely-specified exposure/gain controls, imprecise timestamps, and no trigger support in the class. Machine-vision interfaces (USB3 Vision/GenICam via vendor SDKs) deliver raw pixels, physical-unit controls, per-frame hardware timestamps and trigger I/O. Rule of thumb: UVC is acceptable exactly where rolling shutter is acceptable — static bring-up.

A.4 Optics: M12 vs C/CS mounts

M12 (“S-mount”) lenses are the small threaded board lenses of webcam-class hardware: cheap, fixed aperture, focused by rotating the barrel, no standardized flange distance. C-mount and CS-mount are the machine-vision standards: metal barrels, multi-element glass, iris and focus rings with lock screws, standardized flange distances (C: 17.53 mm; CS: 12.53 mm; a C lens fits a CS body via a 5 mm spacer, not conversely). Three separate data-quality channels:

1. *Sharpness (MTF) is the true floor under σ_d* : sub-pixel localization precision scales with the optical blur radius; a soft lens can double σ_d , and via (13) double σ_Z — equivalent to discarding half the sensor resolution (cf. the weakest-link law of subsection 10.4: the lens is a full member of that chain).
2. *Distortion modelability*: the Brown model (4) absorbs the smooth distortion of decent lenses; cheap wide M12s exhibit higher-order or non-radial terms that leave *systematic* calibration residuals.
3. *Stability of $\mathbf{K}$* : thread-focused plastic barrels creep with vibration and temperature — silent recalibration events, precisely what the $\mathbf{F}$ -drift monitor of section 6 exists to catch.

Locked C/CS rings hold **K** for months. Autofocus is this failure mode as a feature: *a calibrated camera must never refocus*.

An iris adds one more useful knob: most lenses peak in sharpness around $f/4$ – $f/8$; beyond that diffraction itself blurs (spot $\approx 1.22 \lambda N$: at $f/8$ about $5.4 \mu\text{m}$, already exceeding a $3 \mu\text{m}$ pixel).

A.5 Monochrome vs color

A Bayer color sensor samples each color at $\frac{1}{4}$ – $\frac{1}{2}$ of the pixel grid and interpolates the rest; demosaicing smears exactly the high-contrast edges that corners and matching depend on, and costs about one stop of sensitivity. This pipeline processes grayscale throughout, so monochrome sensors deliver strictly better data; color remains at most a redundancy layer in the calibration pattern (subsection 5.6).

A.6 Summary and candidate hardware

Characteristic	Data-quality consequence	When it matters
rolling shutter	~ 0.5 px systematic shear (Example 20)	any motion
no trigger	phantom disparity, ~ 47 mm (Example 21)	dynamic scenes
UVC / MJPEG	structured artifacts in σ_d ; no trigger	beyond bring-up
M12 optics	MTF floor, distortion residuals, K creep	always, silently
C/CS optics	sharp, modelable, mechanically stable K	the metric rig
color (Bayer)	demosaic edge artifacts, -1 stop sensitivity	mono preferred

Table 9: Capture-hardware characteristics and their consequences.

Three procurement tiers follow (indicative 2026 prices):

1. *Bring-up* ($\sim \$150$ /pair): two Logitech C920/C930-class UVC webcams. Rolling shutter, no trigger, but focus and exposure lockable via `v4l2-ctl`; teaches the pipeline every real-world lesson cheaply. Algorithms yes, metric claims no.
2. *Budget metric rig* ($\sim \$150$ – 250 /pair): OV9281-class modules (e.g. Arducam) — 1280×800 *monochrome global shutter*, external-trigger variants, interchangeable M12 optics (choose the best lens, lock it with thread-lock). With a 2.8–4 mm lens, $f \approx 950$ – 1300 px — the regime of all worked examples here. Same idea on CSI: the Raspberry Pi Global Shutter Camera (IMX296, trigger pin, $\sim \$50$).
3. *Solid metric rig* ($\sim \$500$ – 900 /pair): entry machine-vision USB3 (FLIR Blackfly S, Basler ace/dart, or Daheng/HIK equivalents): 1.6–2.3 MP mono global shutter, real trigger I/O, Linux SDKs, C/CS lenses. The tier where the resolution mathematics of subsection 10.3 becomes purchasable.

A packaged shortcut: the Luxonis OAK-D class devices ($\sim \$250$) contain two factory-synchronized global-shutter mono cameras in one USB3 housing — the whole hardware problem in one purchase, at the cost of a fixed ~ 7.5 cm baseline, short for a 4–6 m working volume ($\sigma_Z \propto 1/B$, (13)).

Table 10 lists concrete representatives, sorted by price; prices are indicative mid-2026 street prices per camera and drift, but the *characteristics* are fixed properties of each product.

Model	≈\$	Sensor / resolution	Mono	Global shutter	HW trigger	Raw frames	Lens
Raspberry Pi GS Camera ^a	50	IMX296, 1456 × 1088	— ^a	✓	✓	✓	CS
Logitech C920s ^b	70	1920 × 1080	—	—	—	— ^b	fixed
Arducam OV9281 USB	90	OV9281, 1280 × 800	✓	✓	✓	✓	M12
HIKrobot MV-CS016-10UM	230	IMX273, 1440 × 1080	✓	✓	✓	✓	C
Luxonis OAK-D ^c	250	2×OV9282, 1280 × 800	✓	✓	— ^c	✓	fixed
Basler dart daA1280-54um	280	1280 × 960	✓	✓	✓	✓	S/CS
Intel RealSense D435 ^d	330	2×OV9282 IR	✓	✓	— ^d	✓	fixed
FLIR Blackfly S 16S2M-CS	465	IMX273, 1440 × 1080	✓	✓	✓	✓	CS

Table 10: Representative commercial cameras, sorted by indicative 2026 price per unit. ^aCSI interface (needs a Raspberry Pi, not USB); color filter — monochrome variants from third parties. ^bUVC; uncompressed only at reduced rates, MJPEG otherwise; exposure and focus lockable via `v4l2-ctl`. ^cNo external trigger input, but the internal stereo pair is factory-synchronized — the trigger problem is solved *inside* the box; baseline fixed at 7.5 cm. ^dDepth device; the raw synchronized IR pair is accessible and usable as a stereo source; projector should be disabled for our matcher.

A.7 Blending heterogeneous cameras

Knowing each camera’s characteristics is itself an asset: a *modeled* defect is a correctable or at least gateable one. A rolling-shutter camera whose row-readout time is known can have its rows time-stamped individually and its observations gated out (or motion- compensated) whenever the scene moves; a camera with a measured noise curve enters the triangulation with an honest weight (subsection 10.4); a lens with well-fit distortion contributes unbiased rays while one with known residuals can be trusted only in its image center. The measurement model becomes per-camera, and every characteristic in Table 9 turns from a hidden bias into a term of that model.

A deliberate *blend* of camera types then does more than average — it makes artifacts detectable, because most artifacts are *camera-specific* while the scene is common. MJPEG block edges, rolling shear, demosaic fringes, and specular highlights each live in some cameras and not others (highlights move with viewpoint; compression artifacts sit on one camera’s grid); true structure must be consistent across all views. With identical cameras, a shared systematic error is invisible — every unit votes for the same lie; with heterogeneous hardware the correlation is broken and disagreement *localizes* the artifact, in the same spirit as the **F**-drift and laser-audit health monitors (section 6, subsection 11.2). Complementary pairings fall out naturally: a synchronized global-shutter mono pair as the metric backbone; a high-resolution color camera (rolling shutter tolerated, gated by motion) contributing fine texture and appearance to the model; a wide-FOV unit for coverage and moving-object flagging, its geometry weighted down per subsection 10.4. Geometry from the cameras built for geometry, appearance from the cameras built for appearance, and cross-checks from their disagreement — the *N*-camera analysis of section 10 applies unchanged, since every camera carries its own **K**, distortion, and noise model.

B Root-mean-square (RMS) error: definition and use

Nearly every quantitative claim in this document is an RMS figure. This appendix defines it, explains why it is the right summary here, and maps every way the document uses it.

B.1 Definition

For scalar errors $e_1, \dots, e_N$,

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}, \quad (17)$$

the square root of the mean of the squares. For vector errors (a 2D reprojection offset, a 3D position error) the same formula is applied to the *norms*: $\text{RMS} = \sqrt{\frac{1}{N} \sum \|\mathbf{e}_i\|^2}$, which is also $\sqrt{\sum_k \text{RMS}_k^2}$ over the components — components add *in quadrature*.

Example 22 (RMS vs mean vs max). Errors $\{1, 2, 2, 5\}$: mean = 2.5; $\text{RMS} = \sqrt{(1 + 4 + 4 + 25)/4} = \sqrt{8.5} \approx 2.92$; max = 5. RMS always lies between mean and max, weighting large errors more than the mean does (squaring makes the 5 contribute 25/34 of the total). This document reports RMS as the headline and max separately when tails matter (e.g. [Table 4](#)).

B.2 Why squares

Three reasons, in increasing depth. (i) *Calculus*: squared error is smooth, so least-squares problems have closed forms — every estimator in this system ([Example 4](#) onward) minimizes a sum of squares, and RMS is exactly the quantity those estimators optimize, expressed per sample. (ii) *Probability*: for zero-mean Gaussian noise the RMS of the errors converges to the distribution’s σ ; reporting RMS is reporting the fitted noise scale, directly comparable to the injected σ in every experiment. Least squares *is* maximum likelihood under Gaussian noise, closing the loop. (iii) *Composition*: independent error sources add in quadrature, $\sigma_{\text{tot}}^2 = \sum_j \sigma_j^2$ — the working rule behind the two-coordinate pixel distance, the two-camera angular noise ([16](#)), and the disparity noise $\sigma_d = \sqrt{2} \sigma$ used throughout.

Example 23 (quadrature). Per-coordinate pixel noise $\sigma = 0.3$ px: the Euclidean reprojection distance has RMS $\sqrt{0.3^2 + 0.3^2} = 0.424$ px — the baseline against which the residuals of [subsection 13.2](#) are judged. Disparity, a difference of two independent 0.3 px coordinates, likewise has $\sigma_d = \sqrt{2} \cdot 0.3 \approx 0.42$ px ([Example 13](#)).

B.3 Bias, variance, and what RMS cannot tell you

RMS mixes systematic and random error:

$$\text{RMS}^2 = \underbrace{\bar{e}^2}_{\text{bias}^2} + \underbrace{\frac{1}{N} \sum (e_i - \bar{e})^2}_{\text{variance}}. \quad (18)$$

Example 24 (decomposition). A constant bias of 3 plus zero-mean noise of standard deviation 4 gives $\text{RMS} = \sqrt{3^2 + 4^2} = 5$. Two very different failure modes, one number. This is precisely why the diagnostics of [subsection 13.1](#) never rely on a single RMS: the σ -sweep separates the terms — variance scales with σ , bias does not — and the $\sigma = 0$ run isolates bias exactly.

B.4 Statistical handling

Two facts about RMS as an *estimate*, both load-bearing in [subsection 13.2](#). First, a sample RMS over N Gaussian errors fluctuates with relative standard deviation $\approx 1/\sqrt{2N}$ (χ^2 with N degrees of freedom): with $N = 250$ that is 4.5%, which sets the $\pm 3/\sqrt{2N}$ null bands of the T2/T5 tripwires. Second, residuals of a least-squares fit are *smaller* than the noise, because the fit absorbs p degrees of freedom: $\text{RMS}_{\text{resid}} = \sigma\sqrt{1-p/n}$ for p parameters and n residual coordinates. Triangulation ($p = 3$, $n = 4$) leaves $\sigma\sqrt{1/4} \cdot \sqrt{2} = \sigma/\sqrt{2}$ per-view distance — 0.212 px at $\sigma = 0.3$, matching the measured 0.218 ([Table 2](#)); calibration ($p = 42$, $n = 756$) should leave 0.412 px, and the measured 0.484 is the fired T5 flag of [subsection 13.2](#). An RMS *below* the corrected prediction signals overfitting just as surely as one above signals underfitting.

B.5 Where this document uses RMS

- *Reprojection RMS* (px): residual image distance after triangulation or calibration — the fitted noise scale; compare to $\sqrt{2}\sigma\sqrt{1-p/n}$ ([Table 2](#), [Table 3](#)).
- *3D / position RMS* (m or mm): Euclidean reconstruction or tracking error against ground truth; components reported separately when anisotropy matters (depth vs lateral, [Table 6](#)).
- *Timing RMS* (s or min): event and sunrise timing in the scenario studies ([section 14](#)); same formula, time errors instead of distances.
- *Jitter RMS about an offset*: variance-only RMS after removing a known constant (the sunrise estimator’s offset, [subsection 14.2](#)) — i.e. the second term of (18) alone, used deliberately when the bias is understood and harmless.
- *Predicted RMS*: the error laws (13), (16) are statements about the RMS of infinite repetitions; measured-over-predicted ratios are the consistency statistics of [subsection 13.2](#).

References

- [BM92] Paul J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992.
- [CL96] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of SIGGRAPH 96*, pages 303–312, 1996.
- [DGV⁺20] Jiankang Deng, Jia Guo, Evangelos Ververas, Irene Kotsia, and Stefanos Zafeiriou. RetinaFace: Single-shot multi-level face localisation in the wild. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5203–5212, 2020.
- [DGXZ19] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4690–4699, 2019.
- [FB81] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

- [FP10] Yasutaka Furukawa and Jean Ponce. Accurate, dense, and robust multiview stereopsis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(8):1362–1376, 2010.
- [FTV00] Andrea Fusiello, Emanuele Trucco, and Alessandro Verri. A compact algorithm for rectification of stereo pairs. *Machine Vision and Applications*, 12(1):16–22, 2000.
- [Gen11] Jason Geng. Structured-light 3d surface imaging: a tutorial. *Advances in Optics and Photonics*, 3(2):128–160, 2011.
- [GNH19] Patrick Grother, Mei Ngan, and Kayee Hanaoka. Face recognition vendor test (FRVT) part 2: Identification. Technical Report NISTIR 8271, National Institute of Standards and Technology, 2019.
- [GV13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, fourth edition, 2013.
- [Har97] Richard I. Hartley. In defense of the eight-point algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6):580–593, 1997.
- [Hor87] Berthold K. P. Horn. Closed-form solution of absolute orientation using unit quaternions. *Journal of the Optical Society of America A*, 4(4):629–642, 1987.
- [HRBLM07] Gary B. Huang, Manu Ramesh, Tamara Berg, and Erik Learned-Miller. Labeled faces in the wild: A database for studying face recognition in unconstrained environments. Technical Report 07-49, University of Massachusetts, Amherst, 2007.
- [HS97] Richard I. Hartley and Peter Sturm. Triangulation. *Computer Vision and Image Understanding*, 68(2):146–157, 1997.
- [HZ04] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, second edition, 2004.
- [ISM84] Seiji Inokuchi, Kosuke Sato, and Fumio Matsuda. Range imaging system for 3-d object recognition. In *Proceedings of the International Conference on Pattern Recognition*, pages 806–808, 1984.
- [KBH06] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, pages 61–70, 2006.
- [KS00] Kiriakos N. Kutulakos and Steven M. Seitz. A theory of shape by space carving. *International Journal of Computer Vision*, 38(3):199–218, 2000.
- [Lau94] Aldo Laurentini. The visual hull concept for silhouette-based image understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(2):150–162, 1994.
- [LC87] William E. Lorensen and Harvey E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. In *Proceedings of SIGGRAPH 87*, pages 163–169, 1987.

- [LH81] H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293:133–135, 1981.
- [Low04] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [MAD⁺18] Brianna Maze, Jocelyn Adams, James A. Duncan, Nathan Kalka, Tim Miller, Charles Otto, Anil K. Jain, W. Tyler Niggel, Janet Anderson, Jordan Cheney, and Patrick Grother. IARPA janus benchmark-C: Face dataset and protocol. In *Proceedings of the International Conference on Biometrics*, pages 158–165, 2018.
- [NIH⁺11] Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J. Davison, Pushmeet Kohli, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. KinectFusion: Real-time dense surface mapping and tracking. In *Proceedings of the IEEE International Symposium on Mixed and Augmented Reality*, pages 127–136, 2011.
- [OK93] Masatoshi Okutomi and Takeo Kanade. A multiple-baseline stereo. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 15(4):353–363, 1993.
- [RF18] Joseph Redmon and Ali Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [SCD⁺06] Steven M. Seitz, Brian Curless, James Diebel, Daniel Scharstein, and Richard Szeliski. A comparison and evaluation of multi-view stereo reconstruction algorithms. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 519–528, 2006.
- [SKP15] Florian Schroff, Dmitry Kalenichenko, and James Philbin. FaceNet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 815–823, 2015.
- [SM99] Peter F. Sturm and Stephen J. Maybank. On plane-based camera calibration: A general algorithm, singularities, applications. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 432–437, 1999.
- [SS02] Daniel Scharstein and Richard Szeliski. A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International Journal of Computer Vision*, 47(1–3):7–42, 2002.
- [SSS06] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Photo tourism: Exploring photo collections in 3d. *ACM Transactions on Graphics*, 25(3):835–846, 2006.
- [SWK07] Ruwen Schnabel, Roland Wahl, and Reinhard Klein. Efficient RANSAC for point-cloud shape detection. *Computer Graphics Forum*, 26(2):214–226, 2007.
- [TMHF00] Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. Bundle adjustment — a modern synthesis. In *Vision Algorithms: Theory and Practice*, volume 1883 of *Lecture Notes in Computer Science*, pages 298–372. Springer, 2000.

- [Zha00] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.